ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Punteros

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

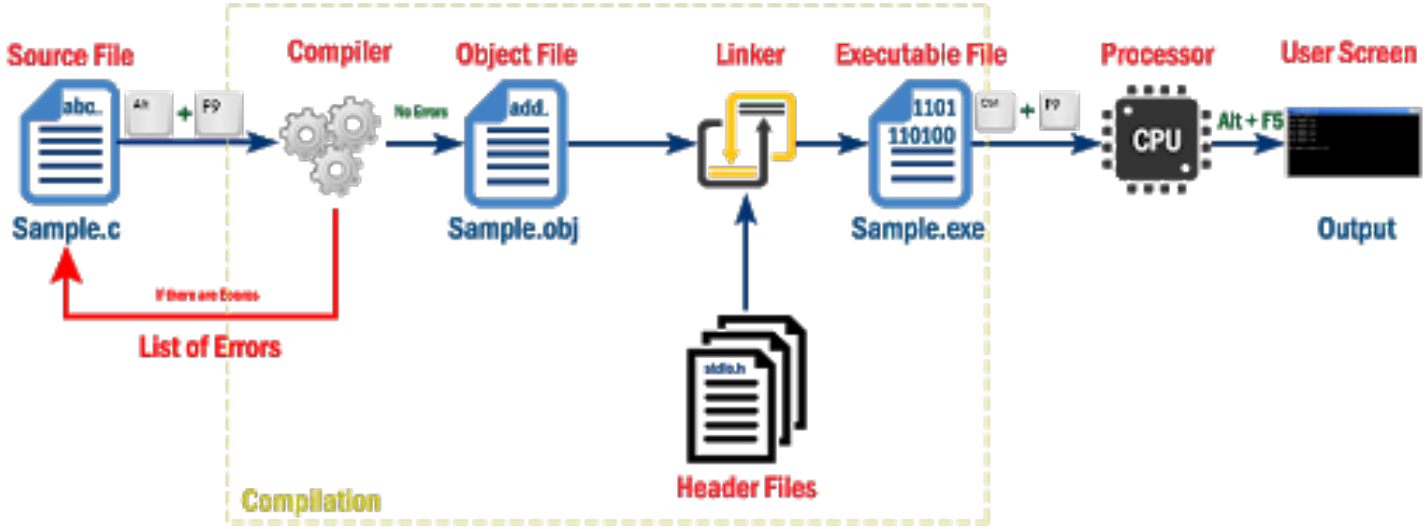
Ing. Civil Telemática - Departamento de Electrónica

2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

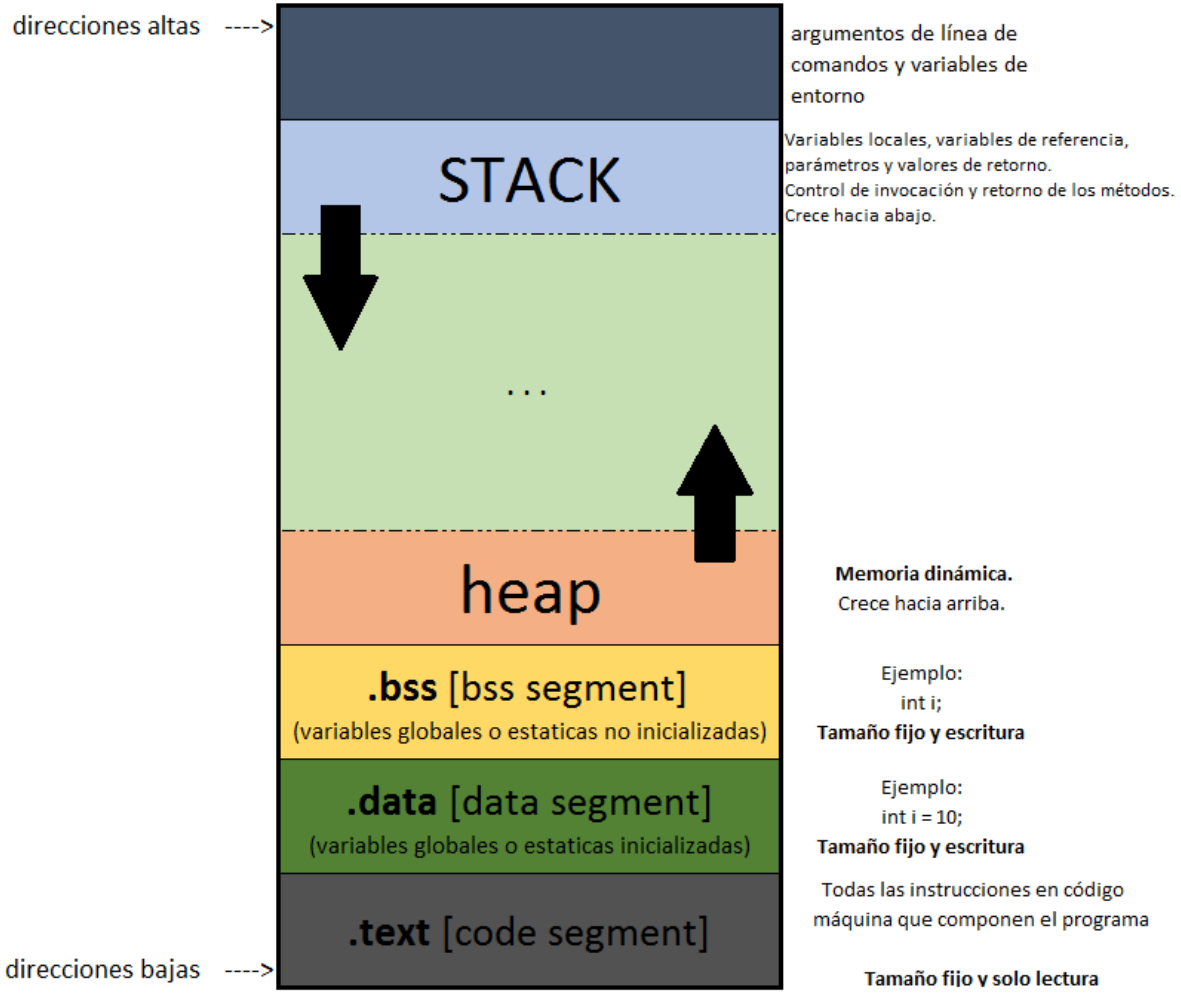
Programa en Memoria

- *Ejecutar*: llevar un programa (binario) a memoria.
- *Variables y Constantes*: espacios de que se asignan y liberan en memoria
- *Ciclo de vida*: cuando se crea y destruye una variable
- *Ámbito*: qué código tiene acceso a la variable



Taxonomía de la memoria

Segmentos de memoria:

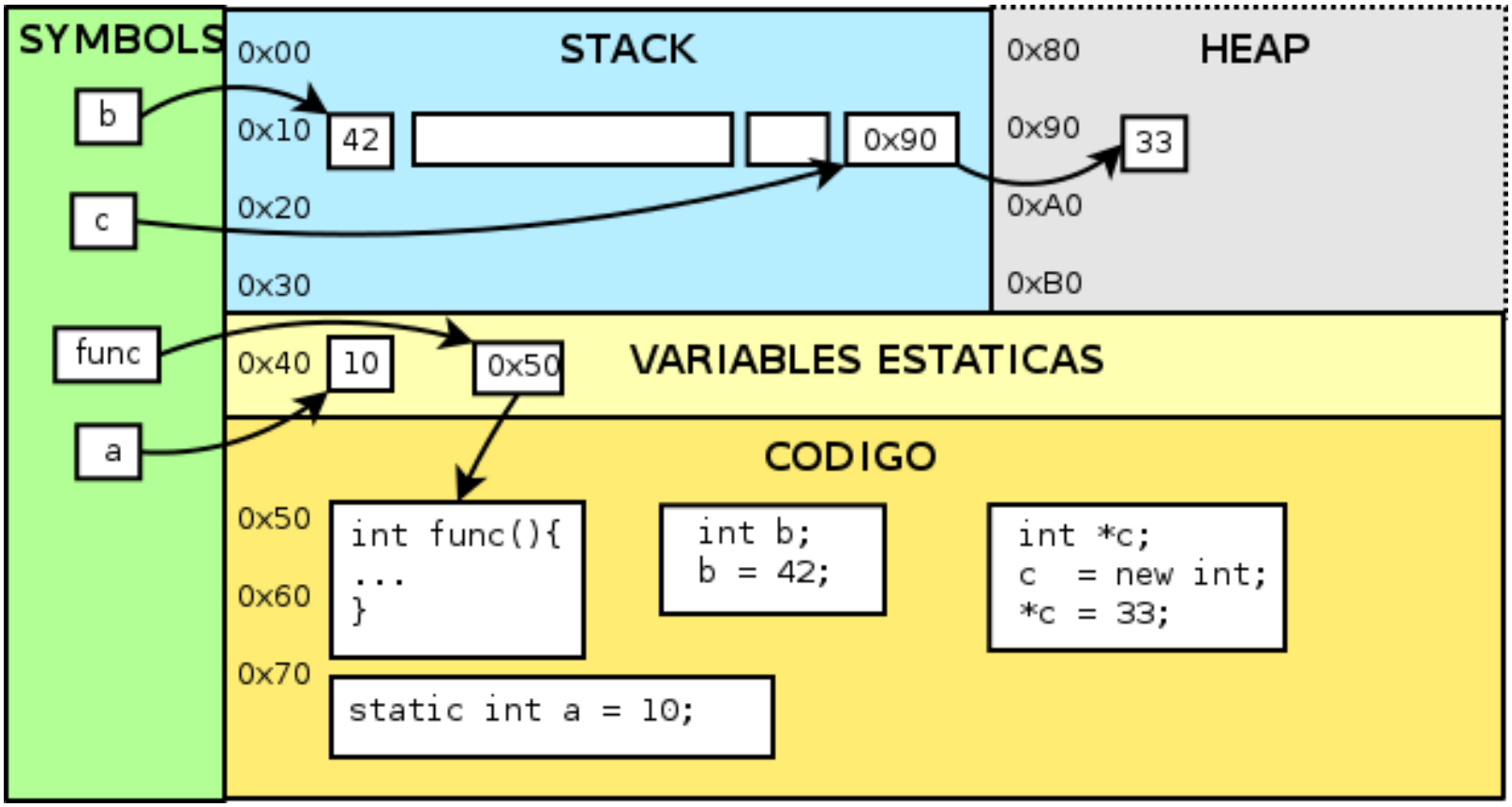


Taxonomía de la memoria

Segmentos de memoria:

- **Segmento de código o text**: donde programa compilado se mantiene en memoria.
- **Memoria estática:** mantiene variables globales y estáticas. Realiza ligado estático de variables en la memoria (hasta que programa termina).
- **Memoria heap:** Permite crear y eliminar objetos de memoria dinámicamente y son referenciadas indirectamente mediante una variable como un **puntero** o una **referencia**.
- **Memoria Stack o call stack:** área dinámica de memoria donde se guardan los parámetros de las funciones, información de las funciones y que permite mantener objetos que se asignan y liberan automáticamente al activar o desactivar el ambiente de ejecución. En C son las variables. Manejado **automaticamente** por el compilador.

Ejemplo


SYMBOLS
b
c
func
a
STACK
0x00
0x10
42
0x90
0x20
0x30
HEAP
0x80
0x90
33
0xA0
0xB0
0x40
10
0x50
VARIABLES ESTATICAS
CODIGO
0x50
0x60
0x70
int func(){
...
}
int b;
b = 42;
int *c;
c = new int;
*c = 33;
static int a = 10;

Dirección de Memoria

C

```c
#include<stdio.h>

int main(){
        int b;
        scanf("%d", &b); //¿Por qué &b?

        return 0;
}
```

- Las variables de stack son cajones que pueden guardar valores, pero ¿dónde están esos cajones?.
- El operador & (dirección de, o referencia) nos entrega la ubicación de memoria donde se encuentra el cajón.

Analogía - Biblioteca

- Va a devolver el libro **¿Cómo pasar EDA sin morir en el intento?**.
- Éste va en el casillero **Problema de Universitarios**.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- `pu = 320;` Bibliotecario (programador) sabé donde está el casillero.
- `scanf("%d", &p);` Lector no sabe donde está el casillero → el bibliotecario le indica.

Analogía - Biblioteca

- `pu = 320;` Bibliotecario (programador) sabé donde está el casillero.
- `scanf("%d", &p);` Lector no sabe donde está el casillero → el bibliotecario le indica.

Analogía - Biblioteca

- `pu = 320;` Bibliotecario (programador) sabé donde está el casillero.
- `scanf("%d", &p);` Lector no sabe donde está el casillero → el bibliotecario le indica.

Variables estáticas

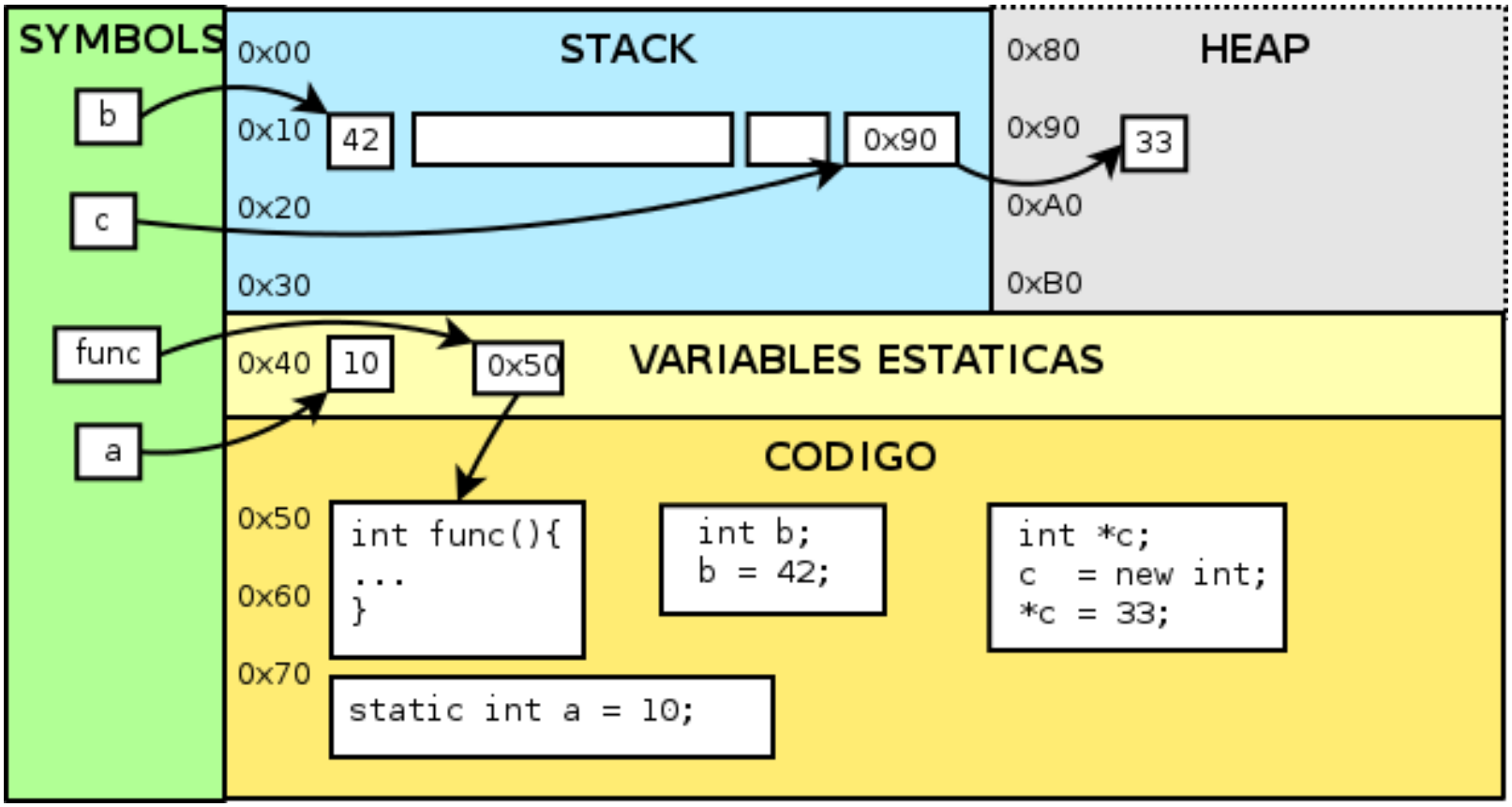
- Ligado a la memoria estática.
- Permanece durante todo el tiempo de ejecución del programa.
- **Ventajas**
  - Útil para definir variables globales y compartidas.
  - Para variables sensibles a la historia de un subprograma (e.g. modificador *static*).
  - Acceso directo permite mayor eficiencia.
- **Desventajas**
  - Impide compartir memoria entre diferentes variables (no se reasigna).

Variables estáticas - static

C

```
long y; //var global.

static int z = 1; //global estatica;
                  //solo visible a este archivo.

int est3_st4tic(){
        static int a=0; //local estatica
        a++; // no pierde su valor al finalizar ámbito
        return a;

}

int fun() {
        int i;
        for(i=0;i<10;i++){
                printf("vuelta numero %d\n", i);
        //      printf("a es imposible de imprimir: %d\n", a);
                printf("pero no pierde su valor: %d\n", est3_st4tic());
        }

        return 0;
}
```

Ejemplo


SYMBOLS
b
c
func
a
STACK
0x00
0x10
42
0x90
0x20
0x30
HEAP
0x80
0x90
33
0xA0
0xB0
VARIABLES ESTATICAS
0x40
10
0x50
CODIGO
0x50
int func(){
...
}
0x60
int b;
b = 42;
int *c;
c = new int;
*c = 33;
0x70
static int a = 10;

Variables dinámicas de stack

- Se asigna memoria dinámicamente; específicamente desde el registro de activación, cuando se ejecuta el ámbito de declaración asociado.
- Se libera cuando termina su ámbito de creación.
- Soportan bien la recursión.
- **Ámbitos de declaración:** procedimento, función, método, bloque.
- En C variables definidas en una función (variables automáticas).

C

```
int fun() {
    int a; // Acá se crea
    {
        float b; //Acá se crea
    }//Acá se destruye b
} // Acá se destruye a
```

Punteros en C

- Variables de stack, que guardan **direcciones de memoria**.
- Peso uniforme en sistemas modernos: 4 bytes in 32-bits, 8 bytes in 64-bits.
- Gestión de memoria dinámica (heap) y direccionamiento (stack/static)
- Se definen con el modificador *, e indicando su tipo.

C

```c
int utfsm1 = 1680; //Dos variables de tipo int.
int utfsm2 = 3939;

int *dir = NULL; //Puntero a un int, parte en NULL
//un puntero guarda la dirección de memoria de una
//variable o segmento de memoria de tipo int.
```

- Diremos que un puntero **apunta** a la dirección de memoria que almacena.

Punteros en C

- Operador de desrreferencia (*): accede al valor en la dirección de memoria **a la que apunta** un puntero.
- Los operadores * y & son opuestos.
- Especificador de punteros: %p.
- *void**: comodín para apuntar a cualquier tipo (cohersión, casting).

C

```c
int utfsm1 = 1680; //Dos variables de tipo int.
int utfsm2 = 3939;

int *dir = NULL;
dir = &utfsm1; //Puntero a un int, apunta a donde está utfsm1

printf("Dirección dentro de dir: %p\n", dir);
printf("Valor dir: %d\n", *dir);
printf("Dirección de dir: %p\n", &dir);
```

- Nota: los punteros también tienen su propia dirección de memoria (existen!).

Punteros en C

- El símbolo * no significa lo mismo dependiendo de si es definición, asignación o implementación.

C

```
float b = 1.0;
float * ptr; // * de declaración, solo indicamos que ptr
             // almacena una dirección de memoria de un float

ptr = &b; // sin *, modificamos la direccion a la que apunta ptr.

*ptr = 5.0; // * de derreferencia, se cambia el valor guardado
            // en la dirección de memoria puntada por ptr

b = *ptr + 1.0; // * de derreferencia, lo que hay guardado donde se apunta + 1.0

float * ptr2 = ptr; // * de declaración.
```

Analogía - Biblioteca

- Preguntamos al bibliotecario o al PC de biblioteca **¿dónde está?** el libro **¿Cómo pasar EDA y FTS al mismo tiempo?**.
- Obtenemos la **dirección de la ubicación** del libro.
- Nos dirigimos a esa dirección $\rightarrow$ **derreferencia**.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Aritmética de Punteros

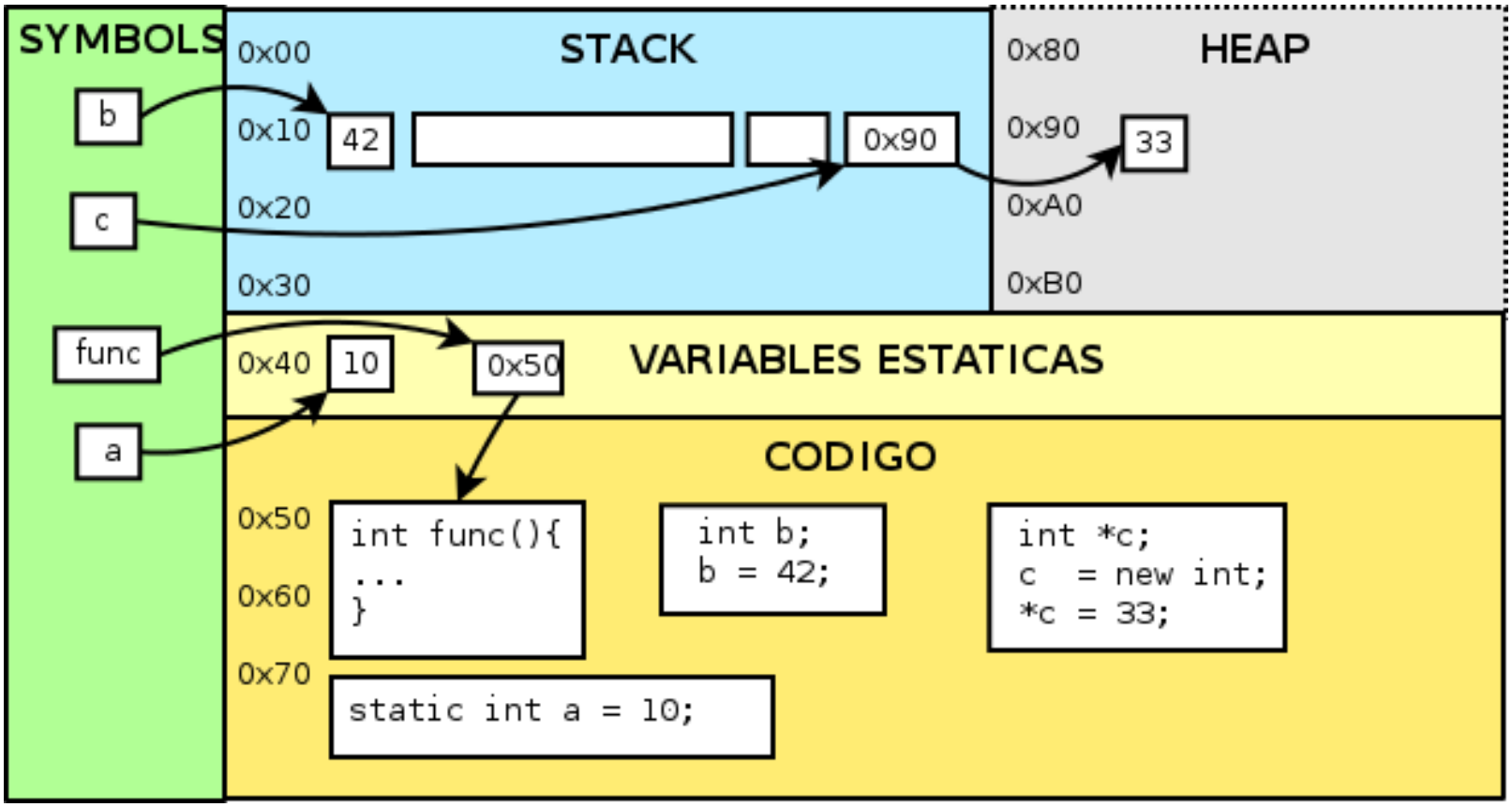
- Indicar el tipo a un puntero sirve para realizar aritmética de punteros.
- Transforma 1 unidad decimal, en 1 unidad segun el tamaño del tipo.
- Podemos saber cuanta memoria hay entre un puntero y el siguiente (si es que existe).
- Ejemplo: Puntero a int avanza de a 4 bytes.

C

```c
float num = 2.5;
float *f;
f = &num; //Dirección a num.

printf("Direccion f: %p\n", f); //¿Cuál es la diferencia
printf("Direccion f+1: %p\n", f+1); //entre f y f+1?
```

- Respuesta: La diferencia es de 4 bytes (tamaño de float).

Ejemplo


SYMBOLS
b
c
func
a
STACK
0x00
0x10
42
0x90
0x20
0x30
HEAP
0x80
0x90
33
0xA0
0xB0
VARIABLES ESTATICAS
0x40
10
0x50
CODIGO
0x50
int func(){
...
}
0x60
int b;
b = 42;
int *c;
c = new int;
*c = 33;
0x70
static int a = 10;

Arreglos y Aritmética de Punteros

- En C un arreglo puede trabajarse con aritmética de punteros.
- Esto se debe a que los arreglos son compatibles con los punteros.
- El compilador hace coerción y esto permite el uso de la aritmética de punteros.

C

```
int a[10], i;
int *pa, *pb;
pa = &a[0]; /* dir. elemento 0 de arreglo a. tamaño 1 int */
pa = a;     /* hace lo mismo que la linea anterior con coerción */
pb = &a;    /* dir. de arreglo a. tamaño 10 int */

for (i=0; i<10; i++){
        printf("%d ", a[i]);
        printf("%d ", *(pa+i));
        printf("%d ", *(a+i));
}
/* los tres for hacen lo mismo */

printf(printf("%d ", *(pb+1)); //¿Qué pasa acá?
```

Valor NULL

- La buena práctica es inicializar los punteros hacia el vacío: el valor especial **nulo** (NULL).

C

```
//Inicializado como puntero nulo:
double *d = NULL;
```

- Aplicaciones de punteros:
  - **Gestión dinámica de memoria:** acceso a la memoria dinámica de heap (limitada por el computador), comparado con el stack (memoria de espacio limitado). Proximamente.
  - **Direccionamiento indirecto:** útil para diseñar estructuras (e.g. listas, árboles o grafos). ELO320 en sí.
  - **Paso por referencia:** pasar elementos por su dirección en memoria, evitando generar una copia al llamar a una función. Ahora!

Funciones y Punteros: Paso por Valor

- Los parametros copian los valores que fueron pasados como parámetros.
- Las variables originales se mantienen inalteradas.

C

```c
#include<stdio.h>

void swap(int a, int b){
        int temp;
        temp = a;
        a = b;
        b = temp;
}

int main(){
        int x=2, y=3;
        swap(x,y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

Funciones y Punteros: Paso por Referencia

- Los parametros son punteros locales, pero la derreferencia es la misma local o global.
- Las variables originales se modifican.

C

```c
#include<stdio.h>

void swap(int *a, int *b){ //punteros locales
        int temp;
        temp = *a; //se guarda lo que hay donde apunta a
        *a = *b; //cambio entre los valores donde apuntan los punteros
        *b = temp; //agrego valor guardado, no se cambian los punteros, ojo.
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

Funciones y Punteros: Paso por Referencia

- ¿Funciona este tercer caso?

C

```c
#include<stdio.h>

void swap(int *a, int *b){ //punteros locales
        int *temp;
        temp = a;
        a = b;
        b = temp;
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

- Respuesta: No, no funciona. Punteros locales.

Funciones y Punteros: Paso por Referencia

- ¿Funciona este cuarto caso?

C

```c
#include<stdio.h>

void swap(int *a, int *b){ //punteros locales
        int *temp;
        temp = a;
        *a = *b;
        *b = *temp;
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

- Respuesta: No, se pierde el valor de *a.